

Lab 8

Introduction to more Advanced Functional Programming with Haskell

Goals

In this lab you will learn to:

1. Use modules in Haskell
2. Use the Haskell REPL
3. Understand patterns in Haskell
4. Use ranges and list comprehensions
5. Make use of various syntax sugar features

Resources

Table 8.1: Haskell Resources

Resource	Link
Learn you a Haskell	http://learnyouahaskell.com/chapters
Haskell base library	https://hackage.haskell.org/package/base-4.14.0.0
Haskell <code>Data.List</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Data-List.html
Haskell <code>Data.Maybe</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Data-Maybe.html
Haskell <code>Data.String</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Data-String.html

Preparations

Skim lectures #1 to #6 for quick recap of Haskell syntax and features. Try to identify features of Haskell that are not present in Elm.

8.1 Meet Haskell

Haskell is often considered a research experiment that escaped the labs: it was mainly designed for teaching and language design research, which is very clearly reflected in their motto:

Avoid success at all costs!

Despite all this, it still gained enough popularity to become an industry-strength language, used by companies like Facebook and Microsoft.

Its main unique feature is *lazy evaluation* and it pioneered *type classes* (type-safe function overloading).

The main Haskell implementation today is the Glasgow Haskell Compiler (GHC), which also includes numerous (optional) *language extensions* to the Haskell 2010 standard.

As a note, Elm's design was inspired by Haskell, so Haskell's syntax should seem familiar.

8.2 The Haskell REPL (GHCi)

To start the Haskell REPL, type `ghci` in your terminal:

```
Haskell REPL
GHCi, version 8.10.7: https://www.haskell.org/ghc/  :? for help
Prelude>
```

To view the type of an expression, we need to use the `:t` command:

```
Haskell REPL
Prelude> :t (+)
(+) :: Num a => a -> a -> a
```

8.3 Modules in Haskell

Question 8.3.1

Open the `Date.elm` file from the 3rd lab and `Date.hs`.
How many differences can you spot?

8.3.1 Exports in Haskell

Exports in Haskell work exactly as in Elm, with a minor syntax difference: instead of the `exposing` keyword we use `where` and place it *after* the list.

Table 8.2: Exports in Elm and Haskell

Export	Elm	Haskell
Export all symbols	<code>module Mod exposing (..)</code>	<code>module Mod where</code>
Export function	<code>module Mod exposing (func)</code>	<code>module Mod (func) where</code>
Export type without its variants	<code>module Mod exposing (Ty)</code>	<code>module Mod (Ty) where</code>
Export type with its variants	<code>module Mod exposing (Ty(..))</code>	<code>module Mod (Ty(..)) where</code>

8.3.2 Imports in Haskell

Table 8.3: Imports in Elm and Haskell

Import	Elm	Haskell
Import all symbols	<code>import Mod exposing (..)</code>	<code>import Mod</code>
Open import	<code>import Mod</code>	<code>import qualified Mod</code>
Qualified import	<code>import Mod as M</code>	<code>import qualified Mod as M</code>
Globally import a type without its variants	<code>import Mod exposing (Ty)</code>	<code>import Mod (Ty)</code>
Globally import a type with its variants	<code>import Mod exposing (Ty(..))</code>	<code>import Mod (Ty(..))</code>
Globally import a function	<code>import Mod exposing (func)</code>	<code>import Mod (func)</code>
Import all except a type or function	N/A	<code>import Mod hiding (Ty, func)</code>

8.3.3 Loading modules in GHCi

Haskell REPL

```
Prelude> :l Date.hs
[1 of 1] Compiling Date           ( Date.hs, interpreted )
Ok, one module loaded.
*Date>
```



Note 8.3.1

When loading modules in the GHCi, visibility rules are **ignored** (i.e. all definitions are visible in the REPL)!



Note 8.3.2

When loading a module GHCi will also load all of its dependencies (imported modules) automatically.



Note 8.3.3

Unlike Elm, Haskell does not automatically reload modules on each function evaluation. To reload all loaded modules use the `:r` command.

8.4 Data definitions in Haskell

8.4.1 Type aliases

Listing 8.4.1: Type alises

Haskell code

```
type Kelvin = Float
type Celsius = Float
```

8.4.2 Type definitions

Listing 8.4.2: Type definitions in Haskell

Haskell code

```
data Color = Red | Green | Blue
data Tree a = Node (Tree a) a (Tree a) | Nil
```

8.4.3 Records

Listing 8.4.3: Records in Haskell

Haskell code

```
data Person = Person {name :: String, age :: Int}
```



Note 8.4.1

In Haskell, records use *nominal* typing, but the functions generated to access fields are placed in module namespace. The consequence, is that if you try to declare two records that have a field with the same name you will get errors.

8.5 Local declarations in Haskell with where

Besides `let ... in`, Haskell also has the `where` keyword for local declarations that are written *after* an expression:

Listing 8.5.1: Local declarations using where

Haskell code

```
double a = d where d = a * a

heron a b c = sqrt (s * (s - a) * (s - b) * (s - c)) where
  s = (a + b + c) / 2

quadruple a = q where
  d = a * a
  q = d * d
```

8.6 Patterns in Haskell

Lets consider the (inefficient) Fibonacci function, first written using only `if` expressions:

Listing 8.6.1 of Patterns.hs (fib)

Haskell code

```
6 fib n =
7   if n == 0 then
8     0
9   else if n == 1 then
10    1
11   else
12    fib (n-1) + fib (n-2)
```

We can also rewrite it using `case` expressions:

Listing 8.6.2 of Patterns.hs (fibCase)

Haskell code

```
29 fibCase n =
30   case n of
31     0 -> 0
32     1 -> 1
33     n -> fibCase (n-1) + fibCase (n-2)
```

8.6.1 Patterns in function definitions

Concept 8.6.1: Refutable and irrefutable patterns

Refutable - patterns that can fail to match (variants of a union type, literals).

Irrefutable - patterns that always match (record fields, data from a type with a single constructor, wildcard patterns, bindings).

Both of the previous methods were also available in Elm, but in Haskell we can also write function definitions with refutable patterns:

Listing 8.6.3 of Patterns.hs (fibPat)

Haskell code

```
16 fibPat 0 = 0
17 fibPat 1 = 1
18 fibPat n = fibPat (n-1) + fibPat (n-2)
```

You can think of this as writing the `case` expression directly in the function definition. This also works for multiple parameters:

Listing 8.6.4 of Patterns.hs (safeDiv)

Haskell code

```
37 safeDiv _ 0 = Nothing
38 safeDiv num d = Just (div num d)
```

8.6.2 Partially-defined functions

By allowing refutable patterns in function definitions and not checking for exhaustiveness, Haskell allows partially defined functions.

Listing 8.6.5 of Patterns.hs (partialHead)

Haskell code

```
109 | partialHead (x:_) = x
```

Haskell REPL

```
> :l Patterns.hs
> partialHead [1, 2 ,3]
1
> partialHead []
*** Exception: <interactive>:1:1-21: Non-exhaustive patterns in function partialHead
```

**Note 8.6.1: Rules for pattern matching and destructuring in Haskell****Variable names must be unique in pattern bindings!****The order of patterns is important!**

→ The patterns are checked from top to bottom, until one pattern matches and that branch is chosen.

→ By default, Haskell won't warn you for redundant patterns!

By default, Haskell does not check for exhaustiveness! If no pattern matches you will get a runtime exception.

8.6.3 Pattern guards

In Haskell we also have *pattern guards*: expressions that can further refine pattern matching results.

Consider the `takeWhile` function, written with the pattern integrated into the function definition:

Listing 8.6.6 of Patterns.hs (takeWhile)

Haskell code

```
42 | takeWhile :: (a -> Bool) -> [a] -> [a]
43 | takeWhile _ [] = []
44 | takeWhile p (x:xs) =
45 |   if p x then
46 |     x : takeWhile p xs
47 |   else
48 |     []
```

We can use pattern guards to also skip the `if` expression, making the code even shorter:

Listing 8.6.7 of Patterns.hs (takeWhileGuard)

Haskell code

```
62 | takeWhileGuard :: (a -> Bool) -> [a] -> [a]
63 | takeWhileGuard _ [] = []
64 | takeWhileGuard p (x:xs)
65 |   | p x = x : takeWhile p xs
66 |   | otherwise = []
```

Pattern guards may also be used with the patterns of `case` expressions:

```

78 | checkDiv num d =
79 |   case safeDiv num d of
80 |     Nothing -> "Div by 0!"
81 |     Just res
82 |       | res == 0 -> "Smaller"
83 |       | otherwise -> "Equal or Greater"

```



Note 8.6.2: Rules for pattern guards

There are 2 main rules for pattern guards:

1. We use `otherwise` as the wildcard (“catch-all”) pattern.
 - By default, guards aren’t checked for exhaustiveness either, so if no guard matches you will get a runtime exception.
2. The guard refinements *are part of the patterns!*
 - In the case of function definitions, we don’t place the `=` after the line with the function definition, but after each guard.
 - In the case of `case` expressions, we don’t place the `->` after the pattern, but after each guard.

Question 8.6.1

Is `otherwise` a language keyword? If not, what is it?

8.6.4 @ pattern bindings

In Haskell when we use patterns for destructuring, we can “have our cake and eat it too” by using @ pattern bindings:

Listing 8.6.9: Example of @ pattern bindings (from Learn You a Haskell) Haskell code

```

capital :: String -> String
capital "" = "Empty string, whoops!"
capital all@(x:xs) = "The first letter of " ++ all ++ " is " ++ [x]

```

The @ pattern binding allows us to access both the “whole” value that is being pattern matched and the resulting bindings (“pieces”) when the match was successful.

8.7 List comprehensions and ranges in Haskell

8.7.1 Ranges

Haskell also has ranges which, as the name suggests, can be used to represent a range of values. For example, we can enumerate all integers between 1 and 10:

Haskell REPL

```

> [1..10]
[1,2,3,4,5,6,7,8,9,10]

```

We can also enumerate every second number between 0 and 15:

```
> [0,2..15]
[0,2,4,6,8,10,12,14]
```

This happens by specifying the first element, the second element according to the a rule you would like and final element.



Note 8.7.1

You can only specify rules that take into consideration the first 2 elements. So if you tried to obtain the powers of 2 with the following code, it wouldn't compile:

```
[1, 2, 4..100]
```

8.7.2 List comprehensions

The other powerful feature for working with lists in Haskell are *list comprehensions*: concise syntax for filtering and transforming lists.

For example consider the `filter` and `map` functions written using list comprehensions:

Listing 8.7.1 of Lists.hs (`filterComp`, `mapComp`)

Haskell code

```
4 | filterComp p l = [x | x <- l, p x]
8 | mapComp f l = [f x | x <- l]
```

We can identify 3 important components of a list comprehension:

$$[\text{map_fn} \mid \text{generator}_1, \dots, \text{generator}_n, \text{filter_exp}_1, \dots, \text{filter_exp}_n]$$

1. The expression that is used to create the elements of the list: `map_fn`
2. Generators: the expression that is used to draw items that will be used:

$$\text{generator}_1, \dots, \text{generator}_n$$

where generator_i has the form $\text{binding}_i \leftarrow \text{generator}_i$

3. Guards: expressions used to filter the generated items:

$$\text{filter_exp}_1, \dots, \text{filter_exp}_n$$

where each filter_exp_i expression returns value of type `Bool`

List comprehensions are not limited to only one list! We can implement the cartesian product of 2 lists using list comprehensions in a very concise way:

Listing 8.7.2 of Lists.hs (`cartesianComp`)

Haskell code

```
12 | cartesianComp lx ly = [(x, y) | x <- lx, y <- ly]
```

Haskell REPL

```
> :l Lists.hs
> cartesianComp [1..3] ['a'..'c']
[(1,'a'),(1,'b'),(1,'c'),(2,'a'),(2,'b'),(2,'c'),(3,'a'),(3,'b'),(3,'c')]
```

8.8 Syntax sugar in Haskell

8.8.1 Infix Operators

In Haskell we can also define infix operators. Let's define an operator `??` for the `Maybe.withDefault` function in Elm which is `fromMaybe` in Haskell:

Listing 8.8.1 of `Sugar.hs` (`??`)

Haskell code

```
4 | infixl 3 ??
5 |
6 | (Just x) ?? _ = x
7 | _ ?? def = def
```

First we declare the operator is infix and left associative with the `infixl` keyword and its precedence (3) (precedence can range from 0 to 9, inclusive, and higher precedence operators are applied first). Then we can write the normal function definition.

8.8.2 Infix call syntax

In Haskell we can call function that take 2 arguments with a special *infix call syntax*, by putting the name of the function between backticks (the ``` symbol, to the left of 1 on your keyboard):

Haskell REPL

```
> div 10 2
5
> 10 `div` 2
5
```

8.8.3 Sections

In Haskell infix operators may also be partially applied, which are called sections:

Haskell REPL

```
> map (+1) [1..10]
[2,3,4,5,6,7,8,9,10,11]
> filter (<4) [1..10]
[1, 2, 3]
> map (10-) [1..10]
[9,8,7,6,5,4,3,2,1,0]
```

8.9 The function application and function composition operators

Remember that in Elm we had the left and right pipe operators `<|` and `|>` and the function composition operators `<<` and `>>`.

In Haskell the equivalent of the pipe right operator (`<|`) is defined as `$` and is called the function application operator and the left function composition operator (`<<`) is `.`.

```

Prelude> sum (map (*2) (filter (\x -> x `mod` 10 + x `div` 10 == 5) [0..10]))
10
Prelude> sum $ map (*2) $ filter (\x -> x `mod` 10 + x `div` 10 == 5) [0..10]
10
Prelude> (\x -> (+1) ((*5) x)) 1
6
Prelude> ((+1) . (*5)) 1
6

```

But since in Haskell we can define infix operators, nothing prevents us implementing them in Haskell:

Listing 8.9.1: Defining Elm’s pipelines and function composition operators in Haskell
Haskell code

```

infixl 0 |>
x |> f = f x

infixr 0 <|
f <| x = f x

infixl 9 >>
f >> g = \x -> g (f x)

infixr 9 <<
f << g = \x -> f (g x)

```

```

Prelude> ((+1) >> (*3) >> (+1)) 1
7
Prelude> [2..100] |> drop 10 |> take 10
[12,13,14,15,16,17,18,19,20,21]

```

8.10 Arbitrary precision integers

To define the non-tail recursive factorial function in GHCi, we need to write it between `:{` and `:}` since it is a multiline function.

```

Prelude> :{
Prelude| fact 0 = 1
Prelude| fact n = n * fact (n - 1)
Prelude| :}

```

Because Haskell has arbitrary precision integers, the only limiting factor for our computations is time and memory:

```
Prelude> fact 1000
40238726007709377354370243392300398571937486421071463254379991042993851239862902
05920442084869694048004799886101971960586316668729948085589013238296699445909974
24504087073759918823627727188732519779505950995276120874975462497043601418278094
64649629105639388743788648733711918104582578364784997701247663288983595573543251
31853239584630755574091142624174743493475534286465766116677973966688202912073791
43853719588249808126867838374559731746136085379534524221586593201928090878297308
43139284440328123155861103697680135730421616874760967587134831202547858932076716
91324484262361314125087802080002616831510273418279777047846358681701643650241536
91398281264810213092761244896359928705114964975419909342221566832572080821333186
11681155361583654698404670897560290095053761647584772842188967964624494516076535
3408198901385442487984959953319101723355566021394503997362807501378376153071277
61926849034352625200015888535147331611702103968175921510907788019393178114194545
25722386554146106289218796022383897147608850627686296714667469756291123408243920
81601537808898939645182632436716167621791689097799119037540312746222899880051954
44414282012187361745992642956581746628302955570299024324153181617210465832036786
90611726015878352075151628422554026517048330422614397428693306169089796848259012
54583271682264580665267699586526822728070757813918581788896522081643483448259932
66043367660176999612831860788386150279465955131156552036093988180612138558600301
43569452722420634463179746059468257310379008402443243846565724501440282188525247
09351906209290231364932734975655139587205596542287497740114133469627154228458623
77387538230483865688976461927383814900140767310446640259899490222221765904339901
88601856652648506179970235619389701786004081188972991831102117122984590164192106
88843871218556461249607987229085192968193723886426148396573822911231250241866493
53143970137428531926649875337218940694281434118520158014123344828015051399694290
15348307764456909907315243327828826986460278986432113908350621709500259738986355
42771967428222487575867657523442202075736305694988250879689281627538488633969099
59826280956121450994871701244516461260379029309120889086942028510640182154399457
15680594187274899809425474217358240106367740459574178516082923013535808184009699
63725242305608559037006242712434169090041536901059339838357779394109700277534720
0000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000
```

Exercise 8.10.1

*

Test in GHCi: `fact 10000` and `fact 100000`. What takes longer, calculating the number or printing all of its digits?



Note 8.10.1

Haskell has a separate type for normal integers (`Int`) and arbitrary precision integers (`Integer`). This is one of the reasons why when you check the signature of `fact` you'll see `fact :: (Eq p, Num p) => p -> p` instead of something like `(fact :: Integer -> Integer)` (more details will follow in later Labs).

8.11 Review questions

Question 8.11.1

*

Enumerate:

- 2 differences between Elm and Haskell
- 1 feature of Elm that are not present in Haskell
- 2 features of Haskell that are not present in Elm

8.12 Practice problems

Exercise 8.12.1

*

Rewrite the `sudan` function from lab 1 using `where` in Haskell. The definition is repeated here for your convenience:

$$S(n, x, y) = \begin{cases} x + y & \text{if } n = 0 \\ x & \text{if } n > 0 \text{ and } y=0 \\ S(n-1, S(n, x, y-1), y + S(n, x, y-1)) & \text{otherwise} \end{cases}$$

Hint:

Try to find a function that is called twice with the same parameters.

Exercise 8.12.2

*

Define an infix operator called `!&` in Haskell, which implements the logical function `nand` (`not and`):

$$x \text{ nand } y = \begin{cases} false & \text{if } x=y=true \\ true & \text{otherwise} \end{cases}$$

Exercise 8.12.3

*

Implement the `safeHead :: [a] -> Maybe a` and `safeTail :: [a] -> Maybe [a]` functions in Haskell which return `Nothing` if the list is empty and the result wrapped in `Just` if it isn't empty.

Haskell REPL

```
> safeHead []
Nothing
> safeHead [1, 2]
Just 1
> safeTail []
Nothing
> safeTail [1, 2]
Just [2]
```

Exercise 8.12.4

*

Write a function called `average :: [Int] -> Float`, which calculates the average of a list of integers.

```
> average [1..10]
5.5
```

Haskell REPL

Hint:

1. Check the signature of the `(/)` operator using the `:t (/)` command in GHCi
2. Use the `fromIntegral` function to convert `Int`s to `Float`s

Exercise 8.12.5

*

Write a function called `countVowels` which counts the number of vowels in a string:

```
> countVowels "lalala"
3
> countVowels "psst"
0
```

Haskell REPL

Exercise 8.12.6

**

Write a function called `addBigs` which adds two big numbers given as lists of digits:

```
> addBigs [1,3,9] [2,2,2]
[3,6,1]
```

Haskell REPL

Hints:

Think about the possible edge cases for this problem. You should make use of the fact that Haskell will let you partially define functions to solve the problem *incrementally*:

1. Write the simplest solution you can come up with, without thinking about the edge cases
2. Test it with a simple input to confirm that it works
3. Try to find an input for which the result is wrong
4. Think about why the result is wrong and go back to step 1 to add code or change the existing code to handle this case
5. Repeat until you can't find more edge cases

Exercise 8.12.7

To format the result of `fact 1000` in $\text{\LaTeX}$, I had to break the number up in groups of 80 digits.

Write a function `breakToLines lineLen str` with the signature `breakToLines :: Int -> String -> [String]` that takes a string as input and returns a list of strings which have length at most `lineLen` characters.

Haskell REPL

```
Prelude> breakToLines 5 (show (fact 20))  
["24329", "02008", "17664", "0000"]
```

Use the `show` function to turn the result of `fact 1000` into a string and write a function `formatLines :: [String] -> String` that takes a list of string, places a newline character between each string and concatenates the list into one string.

Haskell REPL

```
Prelude> formatLines ["First", "Second"]  
"First\nSecond"
```